```python
import sys, os, numpy as np
from am_mcmc import *

def SSq_func(rod, q_current, x_obs, T_obs):
        Phi, h = q_current
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)
        SSq = np.sum((T_obs - T_model)**2)
        return SSq

def gibbs_sigma2(q_current, ns, sigma2_s, n_obs, rod, x_obs, T_obs):
    SSq = SSq_func(rod, q_current, x_obs, T_obs)

    aval = ns/2 + n_obs/2
    bval = (ns * sigma2_s)/2 + SSq/2

    gamma_sample = np.random.gamma(aval, 1/bval)
    return 1 / gamma_sample

def dram(q0, s2_0, J_func,  r_calc, M, ns, sigma2_s, n_obs, rod,  x_obs, T_obs, SSq_func = SSq_func,
         rng_seed=None, k=None):

    if rng_seed is not None:
        np.random.seed(rng_seed)

    p = len(q0)
    q_current = np.array(q0, dtype=float)
    s2_current = s2_0
    samples = np.zeros((M, p + 1))
    samples[0, :p] = q_current
    samples[0, p] = s2_current

    sp = 2.38**2 / p
    gamma2 = 1/25   # delayed rejection cov scaling

    k0 = k

    accepted = 0
    Vk_at_101 = None
    Vk = np.zeros(p)
    for i in range(1, M):

        # ----------------------------
        # Compute AM covariance
        # ----------------------------
        if i < k0:
            q_proposed,D = J_func(q_current, True)

            proposal_cov = D
        else:
            if i == k0:
                Vk = np.cov(samples[:k0, :p], rowvar=False)
                Vk_at_101 = Vk.copy()
            elif np.mod(i, k0) == 1:
                Vk = np.cov(samples[:i, :p], rowvar=False)

            proposal_cov = sp * Vk
            q_proposed = np.random.multivariate_normal(q_current, proposal_cov)

        r_calc = create_posterior_ratio_func(rod, x_obs, T_obs, s2_current)
        # ----------------------------
        # Stage 1 acceptance
        # ----------------------------
        # r_calc = create_posterior_ratio_func(rod, x_obs, T_obs, s2_current)
        # r1 = r_calc(q_proposed, q_current) # dont need sigma, since J is gaussian (it cancels in r( qstar , q^k-1))
        r1 = r_calc(q_proposed, q_current)
        alpha1 = min(1, r1)

        if np.random.rand() < alpha1:
            q_current = q_proposed
            accepted += 1

        else:
            # ----------------------------
            # Delayed Rejection Stage 2
            # ----------------------------

            # if i > k0:
            proposal_cov2 = gamma2**2 * proposal_cov
            q_proposed2 = np.random.multivariate_normal(q_current, proposal_cov2)

            r2 = r_calc(q_proposed2, q_current)

            # Need alpha1(q_proposed2, q_proposed)
            r1_reverse = r_calc(q_proposed, q_proposed2)
            alpha1_reverse = min(1, r1_reverse)

            numerator = r2 * (1 - alpha1_reverse)
            denominator = (1 - alpha1)

            alpha2 = min(1, numerator / denominator) if denominator > 0 else 1

            if np.random.rand() < alpha2:
                q_current = q_proposed2
                accepted += 1

        # Gibbs update for sigma^2
        s2_current = gibbs_sigma2(q_current, ns, sigma2_s, n_obs, rod, x_obs, T_obs)

        samples[i, :p] = q_current
        samples[i, p] = s2_current

    acceptance_rate = accepted / (M - 1)
    print("sapesle", samples.shape)
    return samples, acceptance_rate, Vk_at_101, Vk


def drm(q0, J_func, r_calc, M, rng_seed=None):
    if rng_seed is not None:
        np.random.seed(rng_seed)

    p = len(q0)
    q_current = np.array(q0, dtype=float)
    samples = np.zeros((M, p))
    samples[0] = q_current

    sp = 2.38**2 / p ## only for AM
    gamma2 = 1/25   # delayed rejection cov scaling



    accepted = 0
```

```
Vk_at_101 = None
Vk = np.zeros(p)
for i in range(1, M):


    q_proposed, proposal_cov = J_func(q_current, returnd = True)

    # ----------------------------
    # Stage 1 acceptance
    # ----------------------------
    r1 = r_calc(q_proposed, q_current)
    alpha1 = min(1, r1)

    if np.random.rand() < alpha1:
        q_current = q_proposed
        accepted += 1

    else:
        # ----------------------------
        # Delayed Rejection Stage 2
        # ----------------------------
        proposal_cov2 = gamma2 * proposal_cov
        q_proposed2 = np.random.multivariate_normal(q_current, proposal_cov2)

        r2 = r_calc(q_proposed2, q_current)

        # Need alpha1(q_proposed2, q_proposed)
        r1_reverse = r_calc(q_proposed, q_proposed2)
        alpha1_reverse = min(1, r1_reverse)

        numerator = r2 * (1 - alpha1_reverse)
        denominator = (1 - alpha1)

        alpha2 = min(1, numerator / denominator) if denominator > 0 else 1

        if np.random.rand() < alpha2:
            q_current = q_proposed2
            accepted += 1

    samples[i] = q_current

acceptance_rate = accepted / (M - 1)

return samples, acceptance_rate, Vk_at_101, Vk
```